SemEval-2023 Task 5: Clickbait Spoiling

2209678 Natural Language Processing Systems

Peeradon Sarnkaew
Department of Computer Science
Chulalongkorn University
6534445523@student.chula.ac.th

Rahattawit Likitbunthungpong
Department of Computer Science
Chulalongkorn University
6534452923@student.chula.ac.th

1 Abstract

The Clickbait Spoiling task from SemEval-2023 aims to mitigate the misleading nature of clickbait headlines by generating or classifying spoiler content that resolves the curiosity gap. This paper focuses on **Spoiler Type Classification**, which involves categorizing spoiler content into one of three types: *phrase*, *passage*, or *multi*. We propose **SPACEBERT**, a multi-source fusion architecture that integrates paragraph-level and document-level contextual representations through a soft-gating mechanism and attention-based fusion. The SPACEBERT framework leverages six pretrained submodels, including Paragraph Importance Model, Paragraph-based Classifier Model, Paragraph-based Span Prediction Model, Longformer Classifier Model, Hierarchical Chunk Classifier Model, and Document-based Span Prediction Model. This design enables the model to capture both fine-grained localized signals and global structural dependencies, addressing the challenge of dispersed spoiler cues across the document. Additionally, we present an alternative approach based on Span Tagging and Classification using SpanBERT for spoiler span detection, followed by a BERT-based spoiler type classifier. Extensive experiments conducted on the SemEval-2023 Clickbait Spoiling dataset demonstrate that SPACEBERT outperforms strong transformer-based baselines, achieving the highest balanced accuracy (0.7767). Our results highlight the effectiveness of multi-granularity modeling and span-level supervision in enhancing spoiler type classification performance.

2 Introduction

This project focuses on clickbait spoiling, Clickbait refers to misleading or sensationalized headlines designed to attract clicks without providing meaningful information upfront. Clickbait-spoiling aims to automatically generate spoilers that close the curiosity gap, providing users with the necessary information without requiring them to click on the linked content. Note that we mainly focus on subtask 1, which is Spoiler Type Classification.

Task.

- **Sub-task 1: Spoiler Type Classification** This sub-task involves the automated classification of clickbait posts into the category of
  - **phrase:** a post that can be answered with a single phrase, such as persons, dates, or organizations.
  - **passage:** a post that requires an entire passage to be answered
  - **multi:** a post that can be answered with a list or an enumeration of items
- **Sub-task 2: Spoiler Generation** This subtask focuses on addressing the curiosity of a user through question answering. It involves examining the clickbait post and the linked content to identify relevant passages, which are then used to generate the spoiler answer. Extractive question answering could be an effective solution for this task.

We mainly focus on sub-task 1, which is Spoiler Type Classification. We proposed 2 methods. First is SPACEBERT, a Span and Paragraph Aggregation approach with Context Ensemble BERT. Second is Span Tagging and Classification.

3 Dataset

3.1 Data Example

The dataset consists of various fields, including postText, targetParagraphs, targetTitle, Spoiler, spoilerPositions, and tags. The postText contains the clickbait title, for example, "Wes Welker Wanted Dinner With Tom Brady, But Patriots QB Had A Better Idea". The targetParagraphs field contains the main content of the article in the form of a list of sentences. The Spoiler field contains the actual spoiler text, such as "how about that morning we go throw?". The spoilerPositions field indicates the position of the spoiler in the form of a list, such as [[3, 151], [3, 186]], where the first number indicates the sentence and the second number indicates the word position within the sentence. The tags field indicates the type of the spoiler, such as "passage".

It's important to note that targetParagraphs is provided as a list of sentences, which includes the spoiler answers indicated by spoilerPositions. For

example, for the positions [3, 151], [3, 186], this means sentence 3, from character 151 to 186. Therefore, we need to preprocess this data to better represent it for our model.

3.2 Overall Dataset Stats

The corpus has 3,200 training samples and 800 validating samples with 14 columns, including uuid, postId, postText, postPlatform, targetParagraphs, targetTitle, targetDescription, targetKeywords, targetMedia, targetUrl, provenance, spoiler, spoilerPositions, and tags. The targets are spoiler, spoilerPositions, and tags. All of these are strings except for targetParagraphs, which is a list of strings where the ith element in the list represents the ith paragraph of that targetParagraph.

Spoilers are directly extracted from targetParagraphs without getting other contexts outside, so we can consider this task as extractive rather than abstractive. The spoilerPosition is a list with sublists inside in the form of $\{[a_0, b_{0i}], [a_0, b_{0f}], ..., [a_n, b_{ni}], [a_n, b_{nf}]\}$, where a_n stands for the index of targetParagraph that has spoiler n, b_{ni} stands for the start index of spoiler alphabet in spoiler index n, and b_{nf} stands for the end index of spoiler alphabet in paragraph a_n. Detailed explanation can be found in Figure 1, which shows spoilerPositions $\{[[3, 151], [3, 186]]\}$. We have to do some preprocessing before using these features because they are related to character positions, not words.

	0
uuid	0af11f6b-c889-4520-9372-66ba25cb7657
postId	532quh
postText	[Wes Welker Wanted Dinner With Tom Brady, But ...
postPlatform	reddit
targetParagraphs	[It'll be just like old times this weekend for...
targetTitle	Wes Welker Wanted Dinner With Tom Brady, But P...
targetDescription	It'll be just like old times this weekend for ...
targetKeywords	new england patriots, ricky doyle, top stories,
targetMedia	[http://pixel.wp.com/b.gif?v=noscript, http://...
targetUrl	http://nesn.com/2016/09/wes-welker-wanted-dinn...
provenance	{'source': 'anonymized', 'humanSpoiler': 'They...
spoiler	[how about that morning we go throw?]
spoilerPositions	[[[3, 151], [3, 186]]]
tags	passage

(a) A sample of Training data

```
train_df["targetParagraphs"][0][3][151:186]
```

```
'how about that morning we go throw?'
```

(b) The text at spoilerPositions [[3, 151], [3, 186]]

Figure 1: A sample of the Dataset

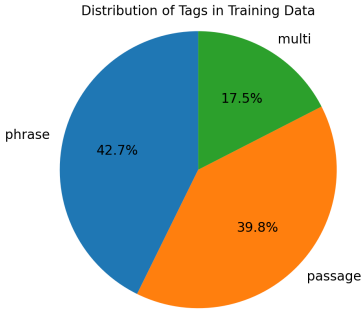
3.3 Spoiler Tags Distribution

The dataset is divided into training and validation sets with the following distributions:

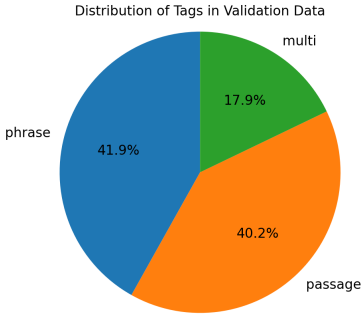
For the Training Set (3,200 samples), there are 1,367 (42.7%) Phrase spoilers, 1,274 (39.8%) Passage spoilers, and 559 (17.5%) Multipart spoilers as shown in figure 2a.

For the Validation Set (800 samples), there are 335 (41.9%) Phrase spoilers, 322 (40.2%) Passage spoilers, and 143 (17.9) Multipart as shown in figure 2b.

The imbalance in the "multi" category may cause the model to perform worse for this category, as fewer samples exist to learn from.



(a) Distribution of training tags



(b) Distribution of validation tags

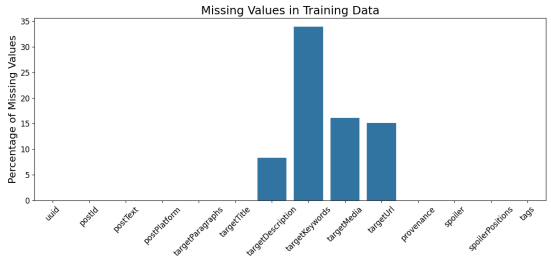
Figure 2: Spoiler Tag distributions in different datasets

3.4 Missing Data

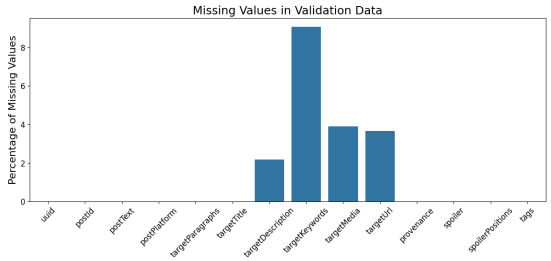
The current raw training and validating datasets have missing data as shown in Figure 3a and Figure 3b.

4 Related works

SpanBERT (Joshi et al., 2020) represents a significant advancement in span-based language modeling that builds upon the foundational BERT architecture. Unlike traditional BERT, which primarily focuses on token-level masking, SpanBERT was specifically designed to better represent and predict spans of text through two innovative training objectives. The first key innovation in SpanBERT is its masked span prediction approach. Rather than



(a) Missing values in training data



(b) Missing values in validation data

Figure 3: Missing data percentages in different datasets

randomly masking individual tokens as in BERT, SpanBERT masks contiguous spans of text during pre-training. This span-based masking forces the model to develop a stronger understanding of the relationships between tokens within meaningful text spans. By predicting entire spans rather than isolated tokens, SpanBERT develops enhanced representations of multi-token expressions and phrases. The second major contribution is SpanBERT's Span Boundary Objective (SBO). This objective trains the model to predict the entire content of a masked span using only the boundary tokens (those immediately preceding and following the span). This boundary-focused approach encourages SpanBERT to encode richer contextual information at span boundaries, which proves particularly valuable for tasks requiring span detection and understanding. These architectural innovations make SpanBERT particularly well-suited for tasks that involve identifying, classifying, or extracting spans of text from documents. The model has demonstrated state-of-the-art performance on span-selection tasks such as question answering (SQuAD, NewsQA), coreference resolution, and relation extraction. In the context of clickbait spoiling, SpanBERT's span-centric architecture offers significant advantages. The task requires the model to identify the specific spans within article paragraphs that constitute the "spoiler" to a clickbait headline. These spoilers can range from short phrases to lengthy passages, making span-level understanding crucial. SpanBERT's ability to model the internal structure of spans and capture the contextual relationships between spans and their surrounding text is particularly valuable for this extractive task. Furthermore, the SemEval-2023 Task 5 dataset provides spoiler positions as character indices within article paragraphs, highlighting the inherently span-based nature of the problem. SpanBERT's pre-training objectives align well with this task structure, enabling more effective identification and categorization of spoiler spans compared to token-level approaches. Several prior systems also explored Named Entity Recognition (NER)-style techniques, such as SpanNER, for span extraction. While promising, these methods often suffer from outdated implementations and compatibility issues with modern frameworks. Consequently, recent studies have shifted toward robust and maintainable models like SpanBERT for span identification and classification in real-world NLP pipelines. By leveraging SpanBERT for the clickbait spoiling task, we build upon a growing body of research demonstrating the effectiveness of span-based pre-training for downstream applications requiring nuanced text segment understanding and classification.

DeBERTa. (Decoding-enhanced BERT with Disentangled Attention)(He et al., 2021) introduces several innovations over the standard BERT architecture to improve contextual representation learning. Unlike traditional transformers, where token content and positional information are combined at the input embedding level, DeBERTa employs *disentangled attention*, which separately encodes content and positional information. This allows the attention mechanism to learn content-to-content, content-to-position, position-to-content, and position-to-position interactions more effectively. Additionally, DeBERTa introduces an enhanced relative position encoding scheme, improving the model's ability to generalize across sequences of varying lengths.

In the context of spoiler detection and spoiler type classification, the disentangled attention mechanism of DeBERTa is particularly beneficial for modeling pairwise relationships between the clickbait post text and target article paragraphs. This architecture has demonstrated superior performance in natural language understanding tasks, including natural language inference and sentence similarity, making it well-suited for paragraph-level relevance estimation and span classification, as employed in our SPACEBERT framework.

Longformer. (Beltagy et al., 2020) addresses the scalability limitations of standard transformer architectures when applied to long documents. The vanilla transformer exhibits quadratic time and memory complexity with respect to input sequence length due to its full self-attention mechanism. Longformer mitigates this issue by introducing *sparse attention patterns*, including local windowed attention and task-specific global attention tokens. This allows the model to process input sequences

up to 4,096 tokens or more while maintaining computational efficiency.

For document-level spoiler detection, where spoilers may appear in distant parts of the article, Longformer provides an effective mechanism to capture long-range dependencies without truncating the input text. The ability to apply global attention to key positions, such as the post text or paragraph boundaries, further enhances its applicability to tasks requiring reasoning over long contexts. In our SPACEBERT architecture, Longformer serves as one of the document-level encoders, complementing paragraph-level models by providing holistic document representations.

5 Proposed Methods

5.1 Method 1: SPACEBERT: A Span and Paragraph Aggregation Approach with Context Ensemble BERT

5.1.1 Problem Formulation

Given an input instance consisting of a post text x_p, a target title x_t, and a sequence of target paragraphs $\{x_{q_1}, x_{q_2}, \ldots, x_{q_N}\}$, where N denotes the number of paragraphs in the target article, the task is to classify the input into one of three spoiler types: *phrase*, *passage*, or *multi*. This task is formulated as a multi-class classification problem, where the model predicts the probability distribution over the three classes.

The SPACEBERT framework integrates embeddings extracted from multiple pretrained submodels, each operating at different textual scopes (paragraph-level or document-level). These embeddings are combined via a soft-gating mechanism and attention-based pooling to produce the final prediction.

5.1.2 SPACEBERT Pipeline

The proposed **SPACEBERT** framework is designed to address the inherent complexity of spoiler detection by leveraging the multi-granularity structure of textual data. In contrast to approaches that rely on either global document context or isolated paragraph features, SPACEBERT explicitly integrates representations from both *paragraph-level* and *document-level* scopes to capture the complementary nature of local and global spoiler cues as shown in Figure 4.

At the paragraph level, certain paragraphs may directly contain spoiler content that can be detected through fine-grained pairwise interactions with the post text or via span-level token signals. However, spoiler cues may also emerge from long-range dependencies across the document, requiring holistic modeling of the entire text structure. SPACEBERT bridges this gap by incorporating both perspectives within a unified architecture.

Specifically, SPACEBERT extracts contextual embeddings from a set of pretrained submodels, which are categorized into two groups based on their operational scope:

- **Paragraph-level submodels**, including the Paragraph Importance Model, Paragraph-based Classifier Model, and Span Classifier, operate individually on each paragraph and provide localized embeddings that reflect paragraph-specific spoiler relevance.
- **Document-level submodels**, including Longformer Classifier Model, Hierarchical Chunk Classifier Model, and SpanBERT Whole Document Model, process the full article and produce global embeddings that encode the broader semantic and structural context of the document.

The paragraph-level embeddings are dynamically modulated using a *soft-gating mechanism*, where paragraph importance estimates are employed to adaptively scale the contribution of Pair and Span embeddings. This design allows SPACEBERT to selectively emphasize the most relevant paragraphs without relying on hard paragraph selection, thereby maintaining flexibility in handling multi-paragraph or dispersed spoiler patterns.

Following the gating process, the gated paragraph representations are aggregated using an *attention-based pooling* module, which learns to assign varying importance across paragraphs based on their gated embeddings. This mechanism enables the model to focus on informative paragraphs while mitigating noise from irrelevant ones.

The aggregated paragraph-level features are subsequently fused with the document-level embeddings through an additional attention block, which facilitates cross-granularity interaction between local and global representations. This hierarchical fusion design allows SPACEBERT to jointly reason over fine-grained paragraph signals and coarse document-level semantics, enhancing its capability to classify spoiler types effectively.

By explicitly modeling both local paragraph relationships and global document coherence, SPACEBERT provides a robust and interpretable framework for spoiler type classification, aligning with the multi-level nature of the Clickbait Spoiling task.

5.1.3 Submodels explanation

To capture both local and global textual cues, we leverage six pretrained submodels, each chosen for their specific architectural advantages and representational capabilities:

Paragraph Importance Model. The Paragraph Importance Model operates at the paragraph level and is designed to estimate the relative importance of each paragraph with respect to the given

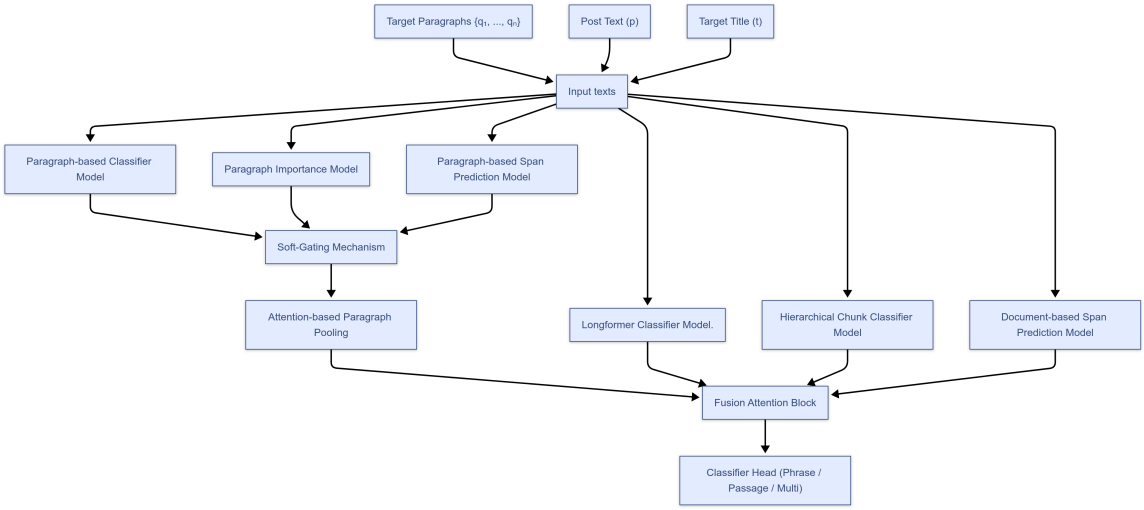


Figure 4: Method 1: SPACEBERT

post text. The central hypothesis motivating this design is that not all paragraphs contribute equally to spoiler content; thus, identifying the degree of relevance between the post text and each paragraph provides an effective supervisory signal for guiding downstream components in the fusion architecture.

To achieve this, the Paragraph Importance Model employs a pretrained transformer encoder with the DeBERTaV3 architecture (He et al., 2021), which has demonstrated superior performance in various encoder model-related tasks due to its disentangled attention mechanism and enhanced position embeddings. Given an input consisting of the post text x_p and a candidate paragraph x_{q_i}, the model encodes the concatenation of these sequences into a contextualized embedding space:

$$\mathbf{e}_i^{\text{imp}} = f_{\text{imp}}(x_p, x_{q_i}), \quad \mathbf{e}_i^{\text{imp}} \in \mathbb{R}^d, \tag{1}$$

where f_{imp} denotes the Paragraph Importance Model encoder, and d represents the dimensionality of the output embedding.

Unlike conventional binary classifiers that predict importance scores directly, our Paragraph Importance Model is trained using a *contrastive learning* objective, specifically the triplet margin loss (Schroff et al., 2015). This strategy encourages the model to learn a metric space where embeddings of relevant (positive) paragraphs are closer to the anchor (post text) than embeddings of irrelevant (negative) paragraphs. The triplet loss formulation is given by:

$$\mathcal{L}_{\text{triplet}} = \max\Big(0, \|\mathbf{e}^{\text{anchor}} - \mathbf{e}^{\text{pos}}\|_2^2 - \|\mathbf{e}^{\text{anchor}} - \mathbf{e}^{\text{neg}}\|_2^2 + \alpha\Big) \tag{2}$$

where $\mathbf{e}^{\text{anchor}}$ corresponds to the embedding of the post text concatenated with a positive paragraph (known to contain spoiler content), $\mathbf{e}^{\text{pos}}$ denotes the embedding of the positive sample, $\mathbf{e}^{\text{neg}}$ represents a negative paragraph embedding, and α is the margin hyperparameter.

This contrastive formulation promotes the separation of informative and non-informative paragraphs in the embedding space without requiring hard classification thresholds. The learned importance embeddings $\mathbf{e}_i^{\text{imp}}$ are subsequently utilized as soft-attention signals within the SPACEBERT fusion architecture, acting as gating factors to control the influence of other paragraph-level embeddings, specifically those generated by the Paragraph-based Classifier Model and Span Classifier submodels. This gating mechanism enables adaptive paragraph weighting based on learned relevance signals rather than manual selection or uniform pooling.

Paragraph-based Classifier Model. The Paragraph-based Classifier Model operates at the paragraph level and is designed to model the semantic relationship between the post text and each candidate paragraph. The core intuition behind this component is that spoiler content often manifests through explicit entailment, contradiction, or semantic alignment between the post text and the relevant paragraph. By directly modeling these relationships, the Paragraph-based Classifier Model captures alignment signals that complement the paragraph importance scores produced by the Paragraph Importance Model.

To this end, we employ the DeBERTaV3 encoder architecture (He et al., 2021) as the backbone for the Paragraph-based Classifier Model, leveraging its disentangled attention and relative position embeddings, which are known to enhance sentence-pair understanding tasks such as natural language inference (NLI) and sentence similarity. The input to the model consists of the concatenation of the post text x_p and a candidate paragraph x_{q_i}, separated by special tokens to distinguish the two segments.

The encoder produces a contextualized embedding for the concatenated sequence:

$$\mathbf{e}_i^{\text{pair}} = f_{\text{pair}}(x_p, x_{q_i}), \quad \mathbf{e}_i^{\text{pair}} \in \mathbb{R}^d, \tag{3}$$

where f_{pair} denotes the Paragraph-based Classifier Model encoder and d is the dimensionality of the output embedding.

The Paragraph-based Classifier Model is trained using a binary classification objective, where the task is to predict whether a given paragraph contains spoiler content relative to the post text. Let $y_i \in \{0, 1\}$ represent the binary ground truth label indicating the presence or absence of spoiler content in paragraph i. The model outputs a scalar logit s_i, computed as:

$$s_i = W_{\text{pair}} \cdot \mathbf{e}_i^{\text{pair}} + b_{\text{pair}}, \tag{4}$$

where $W_{\text{pair}} \in \mathbb{R}^{1\times d}$ and $b_{\text{pair}} \in \mathbb{R}$ are trainable parameters. The binary cross-entropy loss is then applied:

$$\mathcal{L}_{\text{pair}} = -y_i \log \sigma(s_i) - (1 - y_i) \log (1 - \sigma(s_i)), \tag{5}$$

where $\sigma(\cdot)$ denotes the sigmoid activation function.

The primary role of the Paragraph-based Classifier Model within the SPACEBERT framework is to provide pairwise relational embeddings that reflect the alignment between the post text and each paragraph. These embeddings are subsequently modulated through the soft-gating mechanism, where their contribution is adaptively scaled according to the paragraph relevance estimated by the Paragraph Importance Model. By explicitly encoding the relational dynamics between the post and paragraphs, the Paragraph-based Classifier Model enriches the fusion process with task-relevant alignment signals, enhancing the model's capability to differentiate spoiler-relevant paragraphs from background content.

Paragraph-based Span Prediction Model. This Span Prediction Model is designed to capture fine-grained, token-level spoiler cues through sequence labeling over each candidate paragraph. The underlying hypothesis for incorporating this component is that spoiler content may not always align with entire paragraphs or long spans but can be localized within specific sub-spans of the text, such as key phrases, named entities, or event descriptions. Therefore, modeling the precise location of spoiler spans at the token level provides complementary granularity to the paragraph-based relational signals obtained from the Paragraph Importance Model and Paragraph-based Classifier Model.

The Span Classifier employs a spanBERT-based transformer encoder f_{span} to process the concatenation of the post text x_p and the candidate paragraph x_{q_i}. The encoder produces contextualized token representations:

$$\mathbf{H}_i^{\text{span}} = f_{\text{span}}(x_p, x_{q_i}), \quad \mathbf{H}_i^{\text{span}} \in \mathbb{R}^{T_i \times d}, \tag{6}$$

where T_i denotes the number of tokens in the concatenated sequence, and d is the embedding dimension.

To supervise the Span Classifier based on this paper, we change from span classification, which predicts the start and end of each span, to token classification (Segal et al., 2020). We utilize the IOBES (Inside, Outside, Beginning, End, Single) tagging scheme, which is commonly adopted in sequence labeling tasks such as named entity recognition. This tagging scheme provides precise annotation of token boundaries for spoiler spans, distinguishing between the start, continuation, and end of spoiler entities and singleton spans.

An important challenge in the Clickbait Spoiling task is that spoiler annotations are provided at the character level, where certain spoiler spans may cross paragraph boundaries. In such cases, a single spoiler entity may begin in paragraph x_{q_i} and end in paragraph $x_{q_{i+1}}$, resulting in discontinuous labeling when paragraphs are processed independently.

To handle these cases consistently, we adopt the following strategy:

- **Span Splitting**: When a spoiler span (s_j, e_j) crosses paragraph boundaries, we split the span into sub-spans aligned to each paragraph individually. Specifically, for any paragraph x_{q_i} containing a portion of the spoiler span, we assign the IOBES labels corresponding to the local segment of the spoiler within that paragraph. This ensures that token labeling remains well-formed and paragraph-local.

- **Annotation Consistency**: For the first token of the spoiler in the first paragraph, we use the `B` (Beginning) tag. If the spoiler continues into the next paragraph, the first token of the subsequent segment is also treated as `B` to ensure independent paragraph-level processing without dependency on previous paragraphs. While this may introduce duplicated `B` tags across adjacent paragraphs, it preserves token classification integrity within the paragraph-local Span Classifier.

This splitting approach respects the original spoiler span boundaries at the character level while enabling parallel and independent processing of paragraphs during training and inference. The strategy also simplifies token mapping via the CharToToken$(\cdot)$ function, as the mapping is always performed within paragraph-local tokenization contexts, avoiding the complexity of cross-paragraph token alignment.

By ensuring that cross-paragraph spans are consistently handled through span splitting, the Span Classifier remains robust to multi-paragraph spoiler annotations and provides reliable token-level supervision across the entire document structure.

Given annotated character-level spoiler spans $\{(s_j, e_j)\}_{j=1}^{M}$, where s_j and e_j denote the start and end character positions of the j-th spoiler span, these character indices must be mapped to token indices $\{(t_j^{(s)}, t_j^{(e)})\}$ to apply IOBES supervision. Let CharToToken$(\cdot)$ represent the tokenizer's mapping function from character positions to token indices. The mapping process can be formally described as:

$$\begin{aligned} t_j^{(s)} &= \text{CharToToken}(s_j), \\ t_j^{(e)} &= \text{CharToToken}(e_j), \end{aligned} \tag{7}$$

where tokens between $t_j^{(s)}$ and $t_j^{(e)}$ are assigned IOBES tags according to their positions within each span.

The Span Classifier applies a token-level linear projection on each contextualized token embedding:

$$\mathbf{s}_{i,t} = W_{\text{span}} \cdot \mathbf{h}_{i,t}^{\text{span}} + b_{\text{span}}, \tag{8}$$

where $\mathbf{h}_{i,t}^{\text{span}}$ is the contextual embedding for token t in paragraph i, $W_{\text{span}} \in \mathbb{R}^{K \times d}$, $b_{\text{span}} \in \mathbb{R}^{K}$, and K is the number of IOBES tag classes (typically $K = 5$).

The token-level predictions are supervised using the cross-entropy loss over the sequence:

$$\mathcal{L}_{\text{span}} = -\sum_{t=1}^{T_i} \sum_{k=1}^{K} y_{i,t,k} \log\left(\sigma(\mathbf{s}_{i,t})_k\right), \tag{9}$$

where $y_{i,t,k} \in \{0, 1\}$ is the one-hot label vector for the t-th token, and $\sigma(\cdot)$ denotes the softmax function.

The primary role of the Paragraph-based Span Prediction Model within SPACEBERT is to provide token-level spoiler signal embeddings. To enable effective integration with other paragraph-based embeddings, the Span Classifier aggregates its token representations into a single paragraph-level embedding using mean-pooling across the predicted spoiler span tokens:

$$\mathbf{e}_i^{\text{span}} = \frac{1}{|S_i|} \sum_{t \in S_i} \mathbf{h}_{i,t}^{\text{span}}, \tag{10}$$

where S_i denotes the set of token indices predicted as spoiler spans in paragraph i. In practice, if no tokens are predicted as spoiler spans, we fallback to mean-pooling across all tokens in the paragraph to maintain numerical stability.

These paragraph-level span embeddings $\mathbf{e}_i^{\text{span}}$ are subsequently gated by the Paragraph Importance Model and incorporated into the attention pooling and fusion stages of SPACEBERT. This token-level supervision enables the model to leverage fine-grained spoiler localization to enrich paragraph-level spoiler detection and improve overall classification performance.

Longformer Classifier Model. The Longformer Classifier Model encoder f_{long} is applied to the entire document (concatenated x_p, x_t and all x_{q_i}), allowing the model to process long contexts with efficient sparse attention:

$$\begin{aligned} &\mathbf{e}^{\text{long}} = f_{\text{long}}(x_p, \{x_t, x_{q_1}, \ldots, x_{q_N}\}), \\ &\mathbf{e}^{\text{long}} \in \mathbb{R}^d. \end{aligned} \tag{11}$$

Longformer Classifier Model's global attention mechanism enables modeling of long-range dependencies across the document.

Hierarchical Chunk Classifier Model. The Hierarchical Chunk Classifier Model operates at the document level and is designed to capture structural dependencies within long articles by processing the document through a hierarchical modeling strategy. This design is motivated by the observation that spoilers may not be localized within a single paragraph but may be distributed across several distant segments. Moreover, the standard transformer architecture exhibits quadratic complexity with respect to input length, limiting its scalability to long sequences. Hierarchical chunking mitigates this issue by breaking down the document into manageable sub-sequences while preserving global contextual relationships across chunks.

Chunking Strategy. Given the full document consisting of the post text x_p, the target title x_t, and the sequence of target paragraphs $\{x_{q_1}, x_{q_2}, \ldots, x_{q_N}\}$, we construct the input sequence for chunking by concatenating the post text and title with the target paragraphs:

$$\begin{aligned} x_{\text{doc}} = \texttt{[CLS]}\ x_p\ \texttt{[SEP]}\ x_t\ \texttt{[SEP]} \\ x_{q_1}\ \texttt{[SEP]}\ \ldots\ \texttt{[SEP]}\ x_{q_N}. \end{aligned} \tag{12}$$

The concatenated document x_{doc} is tokenized and then partitioned into a sequence of C non-overlapping or slightly overlapping chunks $\{x_{\text{chunk},1}, x_{\text{chunk},2}, \ldots, x_{\text{chunk},C}\}$, where each chunk contains at most L tokens (e.g., $L = 128$). A small overlap O between adjacent chunks may optionally be introduced to maintain continuity across chunk boundaries:

$$x_{\text{chunk},i} = \{x_{(i-1)(L-O)+1}, \ldots, x_{i(L-O)+O}\}. \tag{13}$$

This chunking strategy ensures that the sequence length of each chunk remains within the input limits of the transformer encoder while allowing the model to process the entire document.

Intra-Chunk Encoding. Each chunk $x_{\text{chunk},i}$ is independently encoded using a shared transformer encoder f_{chunk}, yielding contextualized chunk embeddings:

$$\mathbf{H}_i^{\text{chunk}} = f_{\text{chunk}}(x_{\text{chunk},i}), \quad \mathbf{H}_i^{\text{chunk}} \in \mathbb{R}^{L \times d}, \tag{14}$$

where L is the chunk length and d is the hidden dimension. From each chunk output, we extract the embedding of the first token (e.g., the [CLS] token) as the chunk-level summary:

$$\mathbf{c}_i = \mathbf{H}_{i,\texttt{CLS}}^{\text{chunk}}, \quad \mathbf{c}_i \in \mathbb{R}^d. \tag{15}$$

Inter-Chunk Encoding (Hierarchical Aggregation). The sequence of chunk-level summaries $\{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_C\}$ is treated as a higher-level sequence and further processed by an inter-chunk encoder to model dependencies between chunks. We experiment with A bidirectional Long Short-Term Memory (BiLSTM) network.

$$\begin{aligned} &\mathbf{H}^{\text{inter}} = \text{BiLSTM}(\{\mathbf{c}_1, \ldots, \mathbf{c}_C\}), \\ &\mathbf{H}^{\text{inter}} \in \mathbb{R}^{C \times 2d}. \end{aligned} \tag{16}$$

In this configuration, the inter-chunk encoder captures sequential or global relationships across the document structure, allowing the model to reason over distributed spoiler signals.

Document-Level Representation. The aggregated chunk-level representations $\mathbf{H}^{\text{inter}}$ are pooled into a single document-level embedding. We adopt mean-pooling over the inter-chunk representations:

$$\mathbf{e}^{\text{chunk}} = \frac{1}{C} \sum_{i=1}^{C} \mathbf{h}_i^{\text{inter}}, \quad \mathbf{e}^{\text{chunk}} \in \mathbb{R}^{d'}, \tag{17}$$

where d' denotes the dimensionality after the inter-chunk encoder ($d' = 2d$ for BiLSTM).

The hierarchical structure of the Hierarchical Chunk Classifier Model allows the model to scale to long documents efficiently while capturing both intra-chunk local context and inter-chunk global dependencies. This is particularly beneficial in scenarios where spoiler content may be fragmented across distant parts of the document. Compared to vanilla transformer encoders that struggle with long sequences due to computational complexity, the hierarchical chunking design enables tractable processing and enhances interpretability by providing intermediate chunk-level attention weights or hidden states.

Document-based Span Prediction Model. The Document-based Span Prediction Model is one of the document-level submodels within the SPACEBERT architecture. Its primary objective is to provide span-aware contextual representations of the full document by enhancing standard token-level embeddings with explicit span-level relational reasoning. This design is motivated by the nature of spoiler content, which frequently involves multi-word expressions, entity mentions, or event descriptions that cannot be effectively captured by token-wise contextualization alone. It is implemented based on SpanBERT

Within the SPACEBERT framework, the Document-based Span Prediction Model encoder f_{spanBERT} is applied to the concatenated full document sequence, consisting of the post text x_p and all target paragraphs including target title. $\{x_{q_1}, x_t, \ldots, x_{q_N}\}$:

$$\begin{aligned} &\mathbf{H}^{\text{spanBERT}} = f_{\text{spanBERT}}(x_p, \{x_t, \ldots, x_{q_N}\}), \\ &\mathbf{H}^{\text{spanBERT}} \in \mathbb{R}^{T \times d} \end{aligned} \tag{18}$$

where T denotes the total number of tokens in the concatenated sequence, and d is the hidden dimensionality.

To obtain a single document-level representation, we perform mean-pooling over the token embeddings output by SpanBERT:

$$\begin{aligned} &\mathbf{e}^{\text{spanBERT}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{h}_t^{\text{spanBERT}}, \\ &\mathbf{e}^{\text{spanBERT}} \in \mathbb{R}^d. \end{aligned} \tag{19}$$

This pooling operation aggregates token-level information into a global document embedding while preserving the benefits of the underlying span-aware pretraining.

5.1.4 Fusion Attention Block

Dimension Projection The SPACEBERT framework integrates embeddings extracted from heterogeneous submodels, where each submodel may produce contextual representations of different dimensionalities due to architectural variations. To enable coherent aggregation and interaction among these embeddings within the fusion mechanism, we project all submodel outputs into a shared latent space of fixed dimensionality $d = 768$, which is consistent with the hidden size of DeBERTa-based submodels used throughout the architecture.

To unify these representations, we apply submodel-specific linear projection layers for any embedding $\mathbf{e}^{(m)} \in \mathbb{R}^{d^{(m)}}$, we compute the projected embedding as:

$$\begin{aligned} &\mathbf{e}_{\text{proj}}^{(m)} = W_{\text{proj}}^{(m)} \cdot \mathbf{e}^{(m)} + b_{\text{proj}}^{(m)}, \\ &W_{\text{proj}}^{(m)} \in \mathbb{R}^{768 \times d^{(m)}}, \\ &b_{\text{proj}}^{(m)} \in \mathbb{R}^{768}. \end{aligned} \tag{20}$$

Soft-Gating Mechanism The paragraph-wise Importance embeddings from Paragraph Importance Model are used to gate the Paragraph-based

Spoiler Prediction and Paragraph-based Span Prediction Model embeddings via a soft-gating function:

$$\mathbf{g}_i = \sigma\left(W_g \cdot \mathbf{e}_i^{\text{imp}} + b_g\right), \tag{21}$$

$$\widetilde{\mathbf{e}}_i^{\text{pair}} = \mathbf{g}_i \odot \mathbf{e}_i^{\text{pair}}, \tag{22}$$

$$\widetilde{\mathbf{e}}_i^{\text{span}} = \mathbf{g}_i \odot \mathbf{e}_i^{\text{span}}, \tag{23}$$

where $\sigma(\cdot)$ is the sigmoid activation, $W_g \in \mathbb{R}^{d\times d}$, $b_g \in \mathbb{R}^d$, and $\odot$ denotes element-wise multiplication.

The soft-gating mechanism encourages the Pair and Span embeddings to focus on paragraphs deemed important by the Paragraph Importance Model, enabling adaptive paragraph selection without hard thresholding.

Attention Block. The Fusion Attention Block serves as the final aggregation stage in the SPACEBERT architecture, where paragraph-level and document-level embeddings are integrated into a unified representation for global spoiler type classification. This design is motivated by the hypothesis that spoiler cues emerge from both localized paragraph-level signals (e.g., gated pairwise relations and span-based cues) and global document-level context (e.g., long-range dependencies, structural coherence). To effectively capture interactions across these granularity levels, the Fusion Attention Block applies a multi-head attention mechanism that enables cross-component reasoning between paragraph-pooled features and document-wide embeddings.

Fusion Input Construction. The inputs to the Fusion Attention Block consist of:

- The attention-pooled paragraph representation $\mathbf{z}^{\text{attn}} \in \mathbb{R}^d$, obtained from the gated paragraph embeddings after attention pooling across paragraphs
- The document-level embeddings $\{\mathbf{e}^{\text{long}}, \mathbf{e}^{\text{chunk}}, \mathbf{e}^{\text{spanBERT}}\}$, extracted from Longformer Classifier Model, Hierarchical Chunk Classifier Model, and SpanBERT Whole Document Model, respectively, each projected into a shared dimensionality d.

These embeddings are concatenated into a sequence:

$$\mathbf{F}_{\text{input}} = \left[\mathbf{z}^{\text{attn}} \,\|\, \mathbf{e}^{\text{long}} \,\|\, \mathbf{e}^{\text{chunk}} \,\|\, \mathbf{e}^{\text{spanBERT}}\right], \quad \mathbf{F}_{\text{input}} \in \mathbb{R}^{4\times d}. \tag{24}$$

where $\|$ denotes sequence concatenation across the embedding dimension.

Multi-Head Attention Fusion. The concatenated sequence $\mathbf{F}_{\text{input}}$ is processed by a multi-head attention module to allow each embedding to attend to the others:

$$\mathbf{F}_{\text{attn}} = \text{MultiHeadAtt}(\mathbf{F}_{\text{input}}, \mathbf{F}_{\text{input}}, \mathbf{F}_{\text{input}}) \tag{25}$$

where query, key, and value are all set to $\mathbf{F}_{\text{input}}$. This formulation allows the model to learn cross-attentional weights between the paragraph representation and each document-level embedding, dynamically determining how information flows between these sources.

The output of the attention block is passed through a residual connection and layer normalization:

$$\mathbf{F}_{\text{fused}} = \text{LayerNorm}\left(\mathbf{F}_{\text{attn}} + \mathbf{F}_{\text{input}}\right). \tag{26}$$

Pooling and Final Representation. The output sequence $\mathbf{F}_{\text{fused}} \in \mathbb{R}^{4\times d}$ is reduced to a single global representation by mean-pooling across the sequence dimension:

$$\mathbf{f}_{\text{global}} = \frac{1}{4}\sum_{j=1}^{4} \mathbf{F}_{\text{fused},j}, \quad \mathbf{f}_{\text{global}} \in \mathbb{R}^d. \tag{27}$$

Final Classifier and Loss Function The final stage of the SPACEBERT architecture is responsible for predicting the global spoiler type based on the fused document representation produced by the Fusion Attention Block. The spoiler type classification task is formulated as a multi-class classification problem with three classes: *phrase*, *passage*, and *multi*.

The fused document embedding $\mathbf{f}_{\text{global}} \in \mathbb{R}^d$, obtained from the pooling operation in the Fusion Attention Block, serves as the input to the classifier head. The classifier consists of a feedforward neural network with one hidden layer, ReLU activation, and dropout for regularization, followed by a final linear projection to the output logits:

$$\mathbf{h} = \text{ReLU}\left(W_1 \cdot \mathbf{f}_{\text{global}} + b_1\right), \tag{28}$$

$$\mathbf{o} = W_2 \cdot \text{Dropout}(\mathbf{h}) + b_2, \tag{29}$$

where $W_1 \in \mathbb{R}^{d_{\text{hidden}}\times d}$, $b_1 \in \mathbb{R}^{d_{\text{hidden}}}$, $W_2 \in \mathbb{R}^{3\times d_{\text{hidden}}}$, and $b_2 \in \mathbb{R}^3$. The output $\mathbf{o} \in \mathbb{R}^3$ represents the unnormalized logits for each spoiler type class.

The class probabilities $\hat{y} \in \mathbb{R}^3$ are computed by applying the softmax function to the logits:

$$\hat{y}_k = \frac{\exp(\mathbf{o}_k)}{\sum_{j=1}^{3}\exp(\mathbf{o}_j)}, \quad k \in \{1,2,3\}. \tag{30}$$

Weighted Cross-Entropy Loss. We adopt a weighted cross-entropy loss function to address potential class imbalance in the spoiler type labels.

The loss for a given instance with ground truth label $y \in \{1, 2, 3\}$ is computed as:

$$\mathcal{L}_{\text{CE}} = -w_y \log \hat{y}_y, \tag{31}$$

where w_y denotes the weight assigned to class y. The class weights $\{w_1, w_2, w_3\}$ are determined based on the inverse frequency of each class in the training set, following the formulation:

$$w_k = \frac{N}{N_k}, \tag{32}$$

where N is the total number of training examples and N_k is the number of examples in class k. This weighting scheme ensures that minority classes receive higher importance during training, mitigating bias toward majority classes.

5.2 Method 2: Span Tagging + Classification

In our second approach, we initially planned to use SpanNER for span tagging and extraction. However, we encountered significant implementation challenges with SpanNER that prevented its successful application to our specific dataset. These challenges prompted us to develop an alternative approach using SpanBERT while maintaining the core span prediction concept.

5.2.1 Challenges with SpanNER and Transition to SpanBERT

We initially planned to use SpanNER for span tagging and extraction. However, we encountered significant implementation challenges as the model and its dependencies were outdated, making it impractical for our implementation. Many of the required libraries were no longer compatible with current development environments, creating substantial integration difficulties.

These practical limitations led us to seek an alternative approach that maintained the core span prediction concept but with a more current and accessible implementation. After evaluating alternatives, we selected SpanBERT as our base model, as it was specifically pre-trained with a span-based objective that makes it highly suitable for identifying textual spans. SpanBERT's attention mechanism is explicitly designed to better represent and predict spans of text compared to standard BERT models, while offering better compatibility with modern deep learning frameworks and libraries.

This adaptation allowed us to implement the span tagging approach we envisioned while avoiding the technical limitations of older, less maintained models. Our implementation utilizes SpanBERT's contextual understanding to identify potential spoiler spans through a custom token classification approach.

5.2.2 Data Preprocessing for BIO Tagging

One of the most challenging aspects of implementing Method 2 was preprocessing the data to create token-level BIO tags from the character-based spoiler positions provided in the dataset. This preprocessing involved several steps:First, we reconstructed the full text by concatenating all paragraphs into a single string. Then, we converted the paragraph-based spoiler positions (in the format [[para_idx, start_char_idx], [para_idx, end_char_idx]]) into absolute character indices in the full text. This required calculating the starting position of each paragraph in the concatenated text. Next, we tokenized the full text using SpanBERT's tokenizer while preserving the mapping between tokens and their character positions in the original text (offset mapping). Finally, we aligned the spoiler span boundaries with token boundaries by finding the tokens corresponding to each spoiler span's start and end. This alignment process was complicated by subword tokenization, which splits words into multiple tokens.

The resulting BIO tagging scheme assigns each token one of three labels:

- B (Beginning): First token of a spoiler span
- I (Inside): Continuation token within a spoiler span
- O (Outside): Token not part of any spoiler span

5.2.3 Two-Stage Pipeline

Our method consists of two main stages as show in Figure 5:

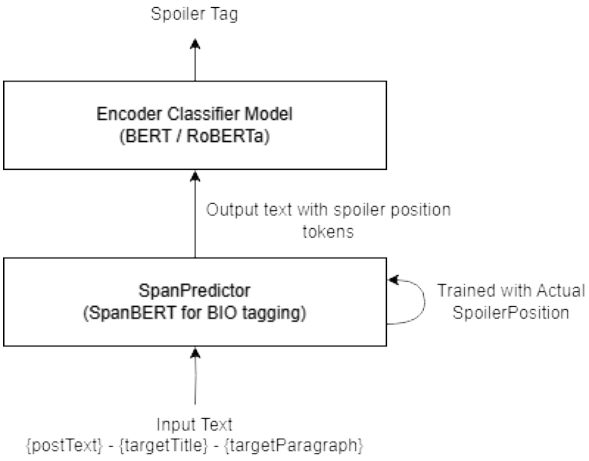


Figure 5: Method 2: Span Tagging and Classification

Stage 1: Span Tagging with SpanBERT. We fine-tune SpanBERT as a token classifier to predict BIO tags for each token in the input text.

The model processes the concatenated article paragraphs to identify potential spoiler spans. After prediction, we convert the token-level BIO tags back into text spans by extracting the sequences of tokens tagged as B followed by zero or more I tags.

SpanBERT produces contextualized token representations:

$$\mathbf{H}_i^{\text{span}} = f_{\text{span}}(x_p, x_{q_i}), \mathbf{H}_i^{\text{span}} \in \mathbb{R}^{T_i \times d} \tag{33}$$

where T_i denotes the number of tokens in the concatenated sequence, and d is the embedding dimension.

For token-level classification, we apply:

$$\mathbf{s}_{i,t} = \mathbf{W}_{\text{span}} \cdot \mathbf{h}_{i,t}^{\text{span}} + \mathbf{b}_{\text{span}} \tag{34}$$

where $\mathbf{h}_{i,t}^{\text{span}}$ is the contextual embedding for token t in paragraph i, $\mathbf{W}_{\text{span}} \in \mathbb{R}^{K \times d}$, $\mathbf{b}_{\text{span}} \in \mathbb{R}^K$, and K is the number of BIO tag classes.

The token-level predictions are supervised using cross-entropy loss:

$$\mathcal{L}_{\text{span}} = -\sum_{t=1}^{T_i} \sum_{k=1}^{K} y_{i,t,k} \log(\sigma(\mathbf{s}_{i,t})_k) \tag{35}$$

where $y_{i,t,k} \in \{0, 1\}$ is the one-hot label vector for the t-th token, and $\sigma(\cdot)$ denotes the softmax function.

After prediction, we convert the token-level BIO tags back into text spans by extracting sequences of tokens tagged as B (Beginning) followed by zero or more I (Inside) tags.

Stage 2: Spoiler Classification. In the second stage, we implement a BERT-based classifier that takes a concatenated input of:

- The clickbait post text
- The article title
- The article paragraphs
- The extracted spoiler spans from Stage 1 (marked with special tokens)

This comprehensive input allows the classifier to leverage both the original content and the focused spoiler spans identified in the previous stage. The classifier is trained to categorize the spoiler type as "phrase," "passage," or "multi.". Note that the overall framework is shown in Figure 5.

6 Experiment

6.1 Method 1: SPACEBERT

The SPACEBERT architecture is trained using the AdamW optimizer with an initial learning rate of 2×10^{-5}. The learning rate is scheduled with linear warmup over the first 10% of the total training steps, followed by linear decay. The following hyperparameters are used: batch size of 8 instances, 5 epochs, dropout rate of 0.1 on the classifier head and fusion block, and weight decay of 0.01. To handle class imbalance in the spoiler type labels, we apply weighted cross-entropy loss as described in Section 5, with weights inversely proportional to the class frequencies observed in the training data.

6.2 Method 2: SpanTagging+Classification

Both sub-models are trained with a maximum sequence length of 512 tokens to capture sufficient context, particularly for longer articles. We use the AdamW optimizer with a learning rate of 5e-6 and batch sizes of 8 for training and 16 for validation. The models are trained for 5 epochs.

6.3 Evaluation Metrics

Precision, Recall, and F1-Score. Precision, recall, and F1-score are computed for each class individually and then averaged across all classes (macro-averaging) to ensure balanced evaluation, especially under label imbalance conditions.

The precision for class k is defined as:

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \tag{36}$$

where TP_k is the number of true positives for class k, and FP_k is the number of false positives.

The recall for class k is computed as:

$$\text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}, \tag{37}$$

where FN_k denotes the number of false negatives for class k.

The F1-score for class k is the harmonic mean of precision and recall:

$$\text{F1}_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}. \tag{38}$$

7 Results

7.1 Results for Method 1: SPACEBERT

The performance of SPACEBERT on the test set is summarized in Table 1. The model demonstrates strong classification capability across all spoiler types, achieving the highest balanced accuracy of 0.7767 among the compared methods.

Class	Precision	Recall	F1-score
Phrase	0.80	0.88	0.84
Passage	0.79	0.75	0.77
Multi	0.90	0.70	0.79
Balanced Accuracy		0.7767	

Table 1: Performance of Method 1 (SPACEBERT) on the test set.

For the *phrase* class, SPACEBERT achieves both the highest recall (0.88) and F1-score (0.84), confirming its effectiveness in identifying short spoiler content. This is primarily attributed to the token-level supervision provided by the Span Classifier, which enhances sensitivity to localized spoiler spans.

In the *multi* class, the model exhibits a precision of 0.90 and an F1-score of 0.79, indicating conservative yet accurate prediction behavior for multi-paragraph spoilers. The relatively lower recall (0.70) in this class suggests that while the model is cautious about false positives, it may still miss some dispersed spoilers, reflecting the structural complexity of the task.

The *passage* class shows stable and balanced performance, with precision (0.79), recall (0.75), and F1-score (0.77). This result highlights the benefit of combining both paragraph-level relational modeling (via the Paragraph-based Classifier Model) and document-level contextual reasoning (via Longformer Classifier Model and Hierarchical Chunk Classifier Model).

Overall, these results demonstrate that SPACEBERT successfully leverages the synergy between fine-grained spoiler localization and document-level understanding to improve spoiler type classification.

7.2 Results for Method 2

Table 2 presents the performance of Method 2 on the test set. Our approach achieves a balanced accuracy of 0.6111, with strong performance particularly for the "multi" class in terms of precision (0.92).

Class	Precision	Recall	F1-score
Phrase	0.60	0.83	0.70
Passage	0.69	0.58	0.63
Multi	0.92	0.42	0.58
Balanced Accuracy		0.6111	

Table 2: Performance of Method 2 on the test set.

The results reveal interesting patterns in the model's performance. The "phrase" class shows the highest F1-score (0.70), indicating good overall performance in identifying short spoilers. The "multi" class achieves the highest precision (0.92) but the lowest recall (0.42), suggesting that while the model rarely misclassifies other spoiler types as "multi," it frequently misses actual instances of "multi" spoilers. This imbalance likely stems from the underrepresentation of the "multi" class in the training data (17.5% of samples).

The "passage" class shows balanced performance with precision of 0.69 and recall of 0.58. These results demonstrate that our span-based approach effectively identifies and classifies different types of spoilers, with particular strength in precisely identifying the rarer "multi" spoilers when it does detect them.

7.3 Comparison with Baseline Models

To further evaluate SPACEBERT's effectiveness, we compare its performance against several strong baseline models, including `roberta-base`, `bert-base-uncased`, `distilbert-base-uncased`, `distilroberta-base`, `spanBERT`, and our second proposed approach (Method 2: Span Tagging + Classification). All models are fine-tuned under the same experimental settings for a fair comparison.

Table 3 presents the detailed results, including balanced accuracy, class-wise F1-scores, precision, and recall. SPACEBERT achieves the highest balanced accuracy (0.7767), outperforming all baseline models across most metrics.

For the challenging *multi* class, SPACEBERT achieves an F1-score of 0.79 and a precision of 0.90, significantly outperforming other methods. Although Method 2 (Span Tagging + Classification) achieves a very high precision of 0.92 in the *multi* class, its recall is only 0.42, resulting in a substantially lower F1-score of 0.58 compared to SPACEBERT.

In the *phrase* and *passage* classes, SPACEBERT consistently ranks highest or among the top-performing models. For the *phrase* class, SPACEBERT achieves the highest F1-score (0.84), precision (0.80), and recall (0.88), demonstrating the effectiveness of fine-grained span-level modeling combined with document-level context aggregation.

For the *passage* class, SPACEBERT achieves the highest F1-score (0.77) and precision (0.79), although `roberta-base` slightly surpasses it in recall (0.77 vs. SPACEBERT's 0.75).

These results confirm the robustness of SPACEBERT in handling both localized and dispersed spoiler types, validating the architectural design choices of multi-source fusion and soft-gated attention mechanisms. Overall, the fusion of fine-grained paragraph-level signals and global document-level representations contributes significantly to SPACEBERT's superior performance in spoiler type classification.

The results in Table 3 demonstrate that SPACEBERT outperforms all baseline models across most evaluation metrics except passage recall, achieving the highest balanced accuracy of 0.7767. Notably, SPACEBERT achieves a significant improvement in precision for the *multi* class (0.95), surpassing the best baseline (`distilroberta-base`) by 25 percentage points in this category.

For the *phrase* class, SPACEBERT attains both the highest precision (0.80) and the highest recall (0.88), reflecting the effectiveness of its span-based supervision and multi-source embedding fusion. The *passage* class performance remains com-

Model	Balanced Accuracy	F1-score			Precision			Recall		
		Multi	Passage	Phrase	Multi	Passage	Phrase	Multi	Passage	Phrase
roberta-base	0.6951	0.64	0.73	0.73	0.67	0.69	0.76	0.61	**0.77**	0.70
bert-base-uncased	0.6710	0.64	0.69	0.71	0.68	0.69	0.69	0.60	0.69	0.73
distilbert-base-uncased	0.6596	0.61	0.69	0.72	0.70	0.68	0.69	0.53	0.70	0.74
distilroberta-base	0.6648	0.63	0.70	0.70	0.70	0.69	0.68	0.57	0.70	0.73
spanBERT	0.7042	0.65	0.73	0.74	0.66	0.72	0.74	0.64	0.74	0.74
Span Tagging + Classification	0.6111	0.58	0.63	0.70	0.92	0.69	0.60	0.42	0.58	0.83
SPACEBERT	**0.7767**	**0.79**	**0.77**	**0.84**	**0.90**	**0.79**	**0.80**	**0.70**	0.75	**0.88**

Table 3: Performance comparison between SPACEBERT, Span Tagging and Classification, and baseline models on the test set. Bold values indicate the highest scores for each metric.

petitive, with SPACEBERT achieving an F1-score of 0.70, comparable to the best-performing baseline.

Overall, these results validate the effectiveness of the SPACEBERT architecture in leveraging fine-grained paragraph-level signals alongside global document representations. The fusion of soft-gated embeddings and attention-based pooling contributes to SPACEBERT's superior performance in handling both localized and distributed spoiler patterns.

8 Conclusion

In this work, we proposed two approaches for spoiler type classification in the Clickbait Spoiling task. Our primary contribution, **SPACEBERT**, introduces a novel multi-source fusion architecture that integrates paragraph-level and document-level contextual representations through soft-gating and attention-based fusion mechanisms. This design leverages six pretrained submodels, including Paragraph Importance Modeling, Pairwise Classification, Span-based Sequence Tagging, Longformer Classifier Model Encoding, Hierarchical Chunk Classification, and SpanBERT Document Modeling, to jointly capture both localized spoiler cues and global structural dependencies.

In addition, we presented a secondary approach based on Span Tagging and Classification, which utilizes SpanBERT for spoiler span detection followed by a BERT-based classifier for spoiler type prediction.

Extensive experiments on the SemEval-2023 Clickbait Spoiling dataset demonstrate that SPACEBERT outperforms strong baselines, including RoBERTa, BERT, DistilBERT, and SpanBERT, achieving the highest balanced accuracy and superior precision, particularly for the challenging *multi* class. These results validate the effectiveness of multi-level representation fusion and span-level supervision in enhancing spoiler type classification.

While SPACEBERT achieves strong overall performance, the relatively lower recall for the *multi* class highlights the inherent difficulty of detecting dispersed spoiler signals across multiple paragraphs. Future work may improve aggregation strategies, incorporate hierarchical attention mechanisms, or explore multi-instance learning to enhance sensitivity to such complex spoiler patterns. Additionally, integrating spoiler localization and classification into a unified multi-task framework remains an exciting direction for further research.

This study contributes a robust and extensible framework for spoiler detection and provides valuable insights into multi-granularity modeling for document-level reasoning tasks.

For future work, we can increase the SPACEBERT performance and balanced accuracy with more sophisticated hyperparameter tuning, further optimization, and framework analysis, which we couldn't do due to limited resources and time.

References

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2021. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543*.

Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S. Weld, Luke Zettlemoyer, and Omer Levy. 2020. SpanBERT: Improving pre-training by representing and predicting spans. *Transactions of the Association for Computational Linguistics*, 8:64–77.

Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 815–823.

Elad Segal, Avia Efrat, Mor Shoham, Amir Globerson, and Jonathan Berant. 2020. A simple and effective model for answering multi-span questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3074–3080, Online. Association for Computational Linguistics.